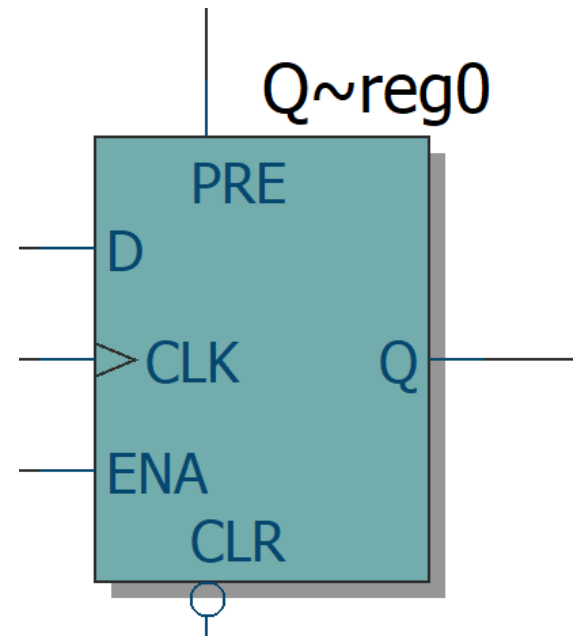


Introducción a VHDL

Matías Javier Oliva
Pablo García

Flip-flop D completo

```
entity ff_d is  
  
  port(  
  
    D: in std_logic;  
    clk: in std_logic;  
    rst_n: in std_logic;  
    set: in std_logic;  
    enable : in std_logic;  
  
    Q: out std_logic  
  
  );  
end entity;
```



¡Conviene tener la entidad a mano para usarla
como componente en distintos diseños!

Testbenches

- Para probar sistemas más complejos establecer manualmente los estados de las señales resulta engorroso.
- En estos casos se usan entidades especiales llamadas testbenches.
- Estas son entidades **sin puertos de entrada ni salida** que instancian los módulos que queremos probar (DUT device under test).
- Dentro de la arquitectura del módulo le damos valores a las señales en momentos de tiempo predefinidos.
- Tener en cuenta que esto no es algo sintetizable, sirve para simulación únicamente.
- La escritura de testbenches puede complicarse significativamente. En este curso veremos algunos ejemplos sencillos.

Testbenches: Sintaxis

```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity testbench is  
end entity testbench;  
  
architecture test of testbench is  
    --Codigo del testbench  
end test;
```

Entidad vacía

Código no
sintetizable

Asignación de variables

Opciones de variable time

fs	$10^{-15} s$
ps	$10^{-12} s$
ns	$10^{-9} s$
us	$10^{-6} s$
ms	$10^{-3} s$
sec	$1 s$
min	$60 s$
hr	$3600 s$

Sentencia AFTER:

```
<signal> <= <initial_value>, <end_value> after <time>;
```

- Es una asignación (por fuera de process)
- Sirve para asignarle un valor a una señal en un momento específico en el futuro.
- Por ejemplo:


```
reset <= '1', '0' after 1 us;
```
- También puede usarse sin valor inicial para programar continuamente cambios en la señal.
- Esto resulta muy útil para generar relojes

```
clock <= not clock after 10 ns;
```

Asignación de variables

Sentencia WAIT:

- Sirve para interrumpir la ejecución de un proceso durante un tiempo determinado.
- Solo puede usarse en procesos que no tengan lista de sensibilidad.
- Existen tres casos para la sentencia WAIT:

```
wait for <time>;
```

Espera una determinada cantidad de tiempo

```
wait until <condition> for <time>;
```

Espera hasta que se cumpla una condición

```
wait on <signal_name>;
```

Espera hasta que haya algún cambio en alguna señal (funciona igual que una lista de sensibilidad)

Testbench: Ejemplo completo

```
library ieee;
use ieee.std_logic_1164.all;
```

```
entity testbench is
end entity testbench;
```

```
architecture test of testbench is
```

```
component Clase3_d is
```

```
port
```

```
(
```

```
-- Input ports
```

```
clk    : in  std_logic;
```

```
dat    : in  std_logic;
```

```
-- Output ports
```

```
out1   : out std_logic
```

```
);
```

```
end component;
```

```
signal clock    : std_logic := '0';
```

```
signal datos    : std_logic := '0';
```

```
signal salida   : std_logic;
```

DUT

Sentencia
AFTER

```
begin
```

```
u0: Clase3_d port map (clock,datos,salida);
```

```
clock <= not clock after 10 ns;
```

```
process
begin
```

```
wait for 40 ns;
```

```
datos <= '1';
```

```
wait for 40 ns;
```

```
datos <= '0';
```

```
end process;
```

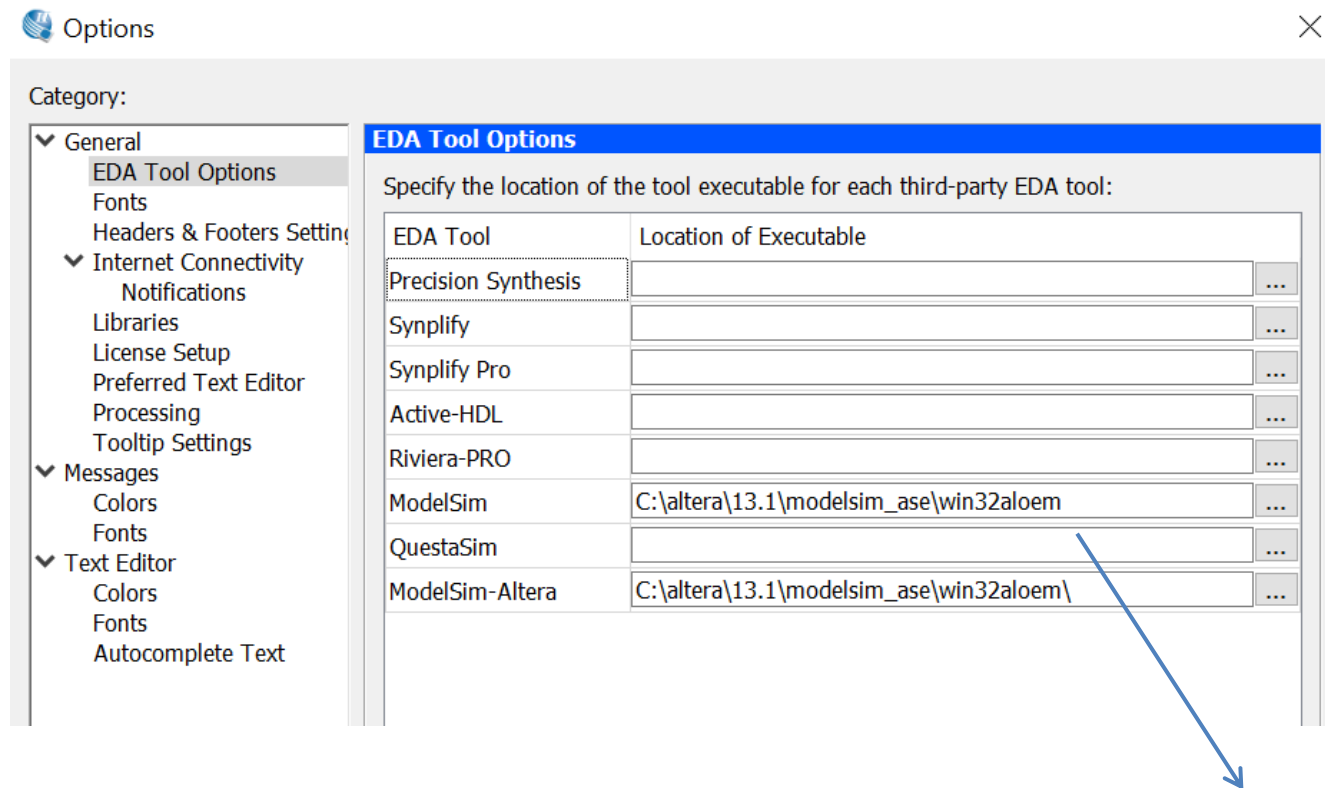
```
end test;
```

Señales
internas

Sentencia
WAIT

Testbench: Configuraciones

Tools -> Options



Especificar path. A veces hay que poner (o sacar) la barra del final

Testbench: Configuraciones

Assignments -> Settings

Category: Device...

- General
- Files
- Libraries
- Operating Settings and Conditions
 - Voltage
 - Temperature
- Compilation Process Settings
 - Early Timing Estimate
 - Incremental Compilation
 - Physical Synthesis Optimization
- EDA Tool Settings
 - Design Entry/Synthesis
 - Simulation**
 - Formal Verification
 - Board-Level
- Analysis & Synthesis Settings
 - VHDL Input
 - Verilog HDL Input
 - Default Parameters
- Fitter Settings
- TimeQuest Timing Analyzer
- Assembler
- Design Assistant
- SignalTap II Logic Analyzer
- Logic Analyzer Interface
- PowerPlay Power Analyzer Settings
- SSN Analyzer

Simulation

Specify options for generating output files for use with other EDA tools.

Tool name: ModelSim-Altera

☐ Run gate-level simulation automatically after compilation

EDA Netlist Writer settings

Format for output netlist: Verilog HDL Time scale: 1 ps

Output directory: eDrive/Documentos/00-Programas_curso_vhdl/Clase3_d/simulation/qsim/ ...

☐ Map illegal HDL characters ☐ Enable glitch filtering

Options for Power Estimation

☐ Generate Value Change Dump (VCD) file script Script Settings...

Design instance name:

More EDA Netlist Writer Settings...

NativeLink settings

☐ None

☒ Compile test bench: tb Test Benches...

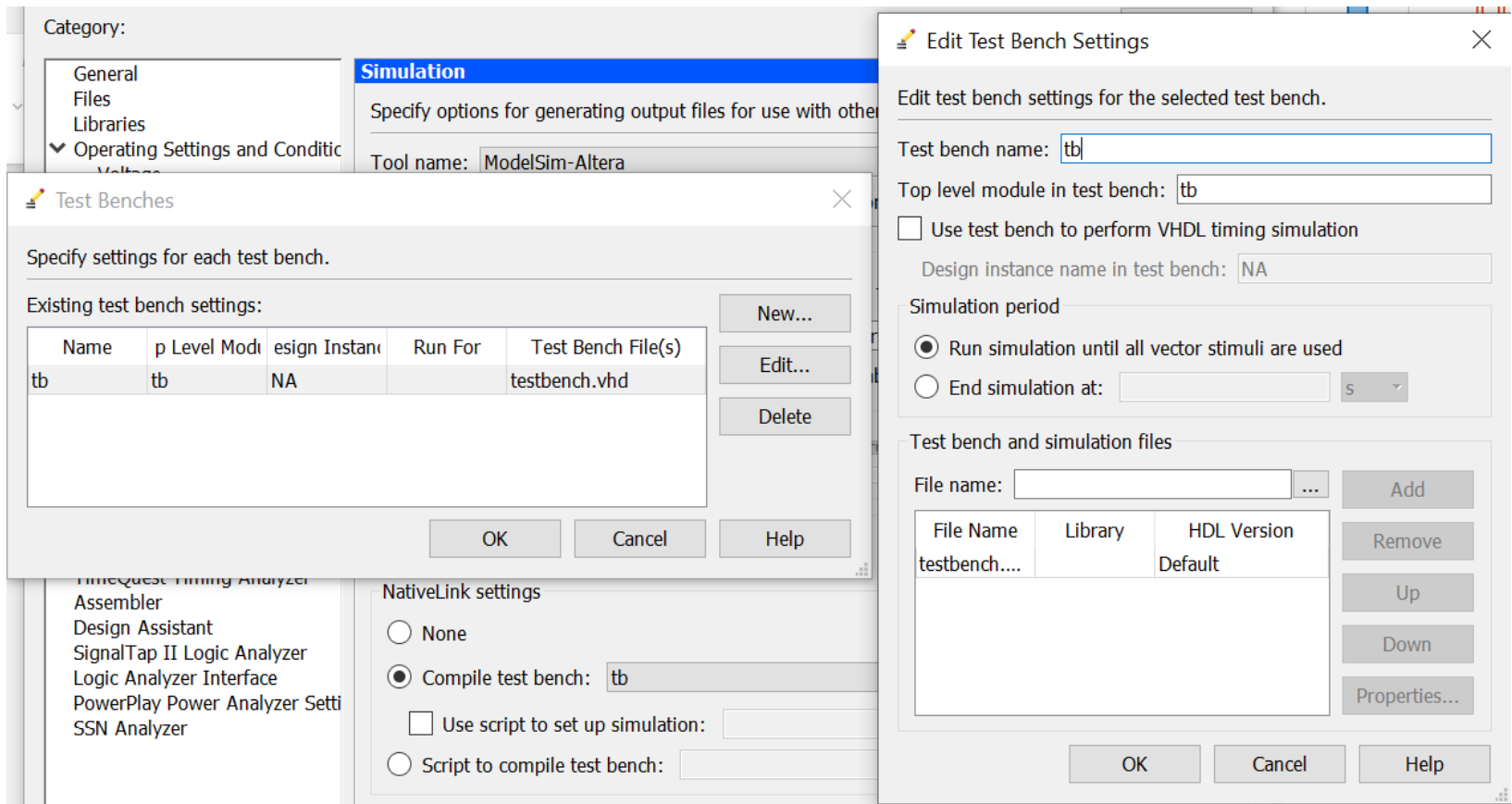
☐ Use script to set up simulation: ...

☐ Script to compile test bench: ...

More NativeLink Settings... Reset

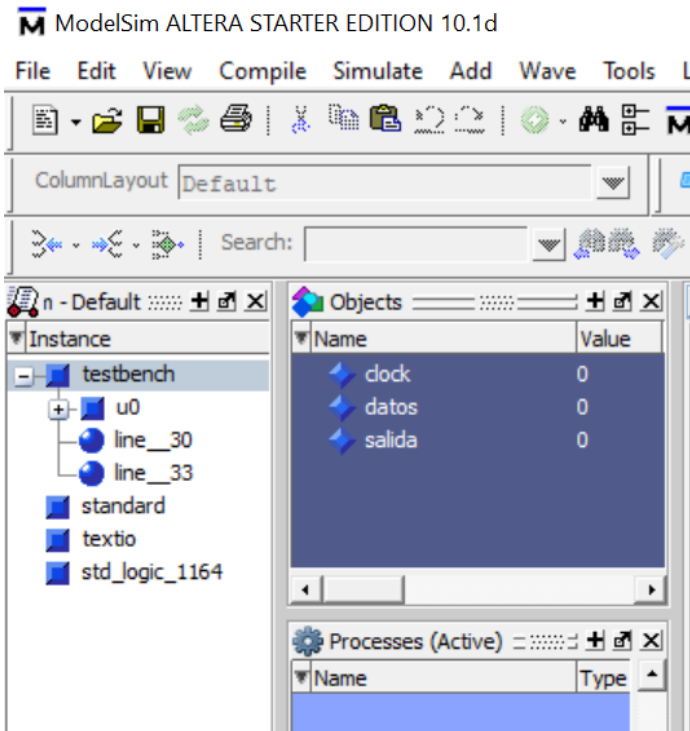
Testbench: Configuraciones

Assignments -> Settings

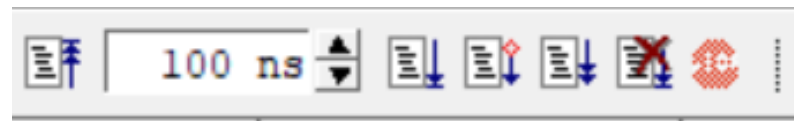


Librerías:

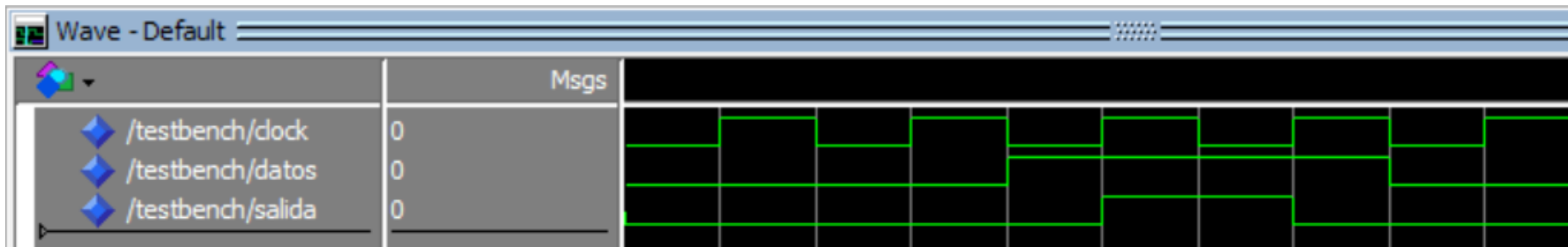
ModelSim ALTERA



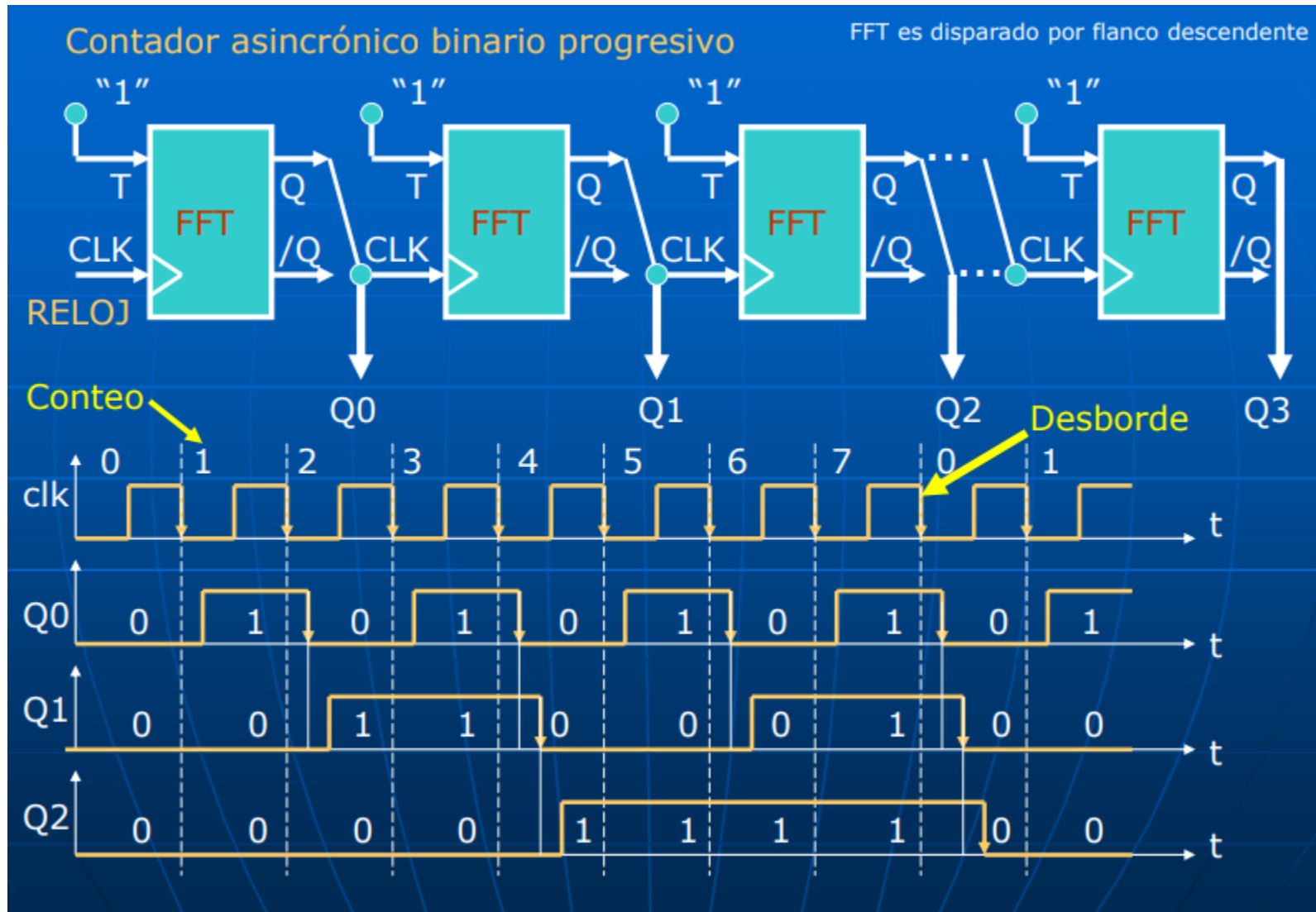
Controles de flujo:



Simulación:



Contadores asincrónicos



Contadores asincrónicos

- Estos contadores son sencillos de implementar pero tienen algunos inconvenientes:
 - Si los flip-flop no son ideales se producen códigos inválidos en algunas transiciones del conteo (ver por ejemplo que entre el 1 y el 2 hay un breve 0 que puede ser un problema en algunos casos)
 - La señal de clock sufre retardos provocados por el tiempo de clock a q de cada flip-flop.
 - El último FF define la máxima velocidad a la que se puede operar el contador sin errores
- Un contador así puede contar hasta 2^{N-1}
- Para contar hasta un número distinto (menor a 2^{N-1}) puede aprovecharse las señales de reset asincrónico de los FF

$$f_{max} = \frac{1}{N t_{clk-q}}$$

Implementación en VHDL

```

library ieee;
use ieee.std_logic_1164.all;

entity Clase4_a is
port(
    clk : in std_logic;
    reset : in std_logic;

    count : out std_logic_vector (3 downto 0)
);
end entity;

```

Usamos como componente el
flip-flop tipo T activo en
flancos descendentes del reloj

```

architecture count of Clase4_a is
component FF_T is
    Port (
        T, CLK, SET, RESET : in STD_LOGIC;
        Q, Q_N              : out STD_LOGIC
    );
end component FF_T;

signal count_aux : std_logic_vector (3 downto 0);

begin

ff_t_0 : FF_T port map ('1', clk, '0', reset, count_aux(0), open);
ff_t_1 : FF_T port map ('1', count_aux(0), '0', reset, count_aux(1), open);
ff_t_2 : FF_T port map ('1', count_aux(1), '0', reset, count_aux(2), open);
ff_t_3 : FF_T port map ('1', count_aux(2), '0', reset, count_aux(3), open);

count <= count_aux;
end count;

```

Generalización a N bits

```

architecture count of Clase4_a is

    component FF_T is
        port (
            T, CLK, SET, RESET : in  std_logic;
            Q, Q_N              : out std_logic
        );
    end component;

    signal count_aux : std_logic_vector(N-1 downto 0);

begin

    FF_T_0 : FF_T
        port map (
            T      => '1', CLK      => clk, SET      => '0', RESET => reset,
            Q      => count_aux(0), Q_N      => open
        );

    gen_ff: for i in 1 to N-1 generate
    begin
        FF_T_inst : FF_T
            port map (
                T      => '1', CLK      => count_aux(i-1), SET      => '0',
                RESET => reset, Q      => count_aux(i), Q_N      => open
            );
    end generate;

    count <= count_aux;

end architecture;

```

Testbench

```

library ieee;
use ieee.std_logic_1164.all;

entity testbench is end entity;

architecture arch of testbench is

constant N : integer := 16;

    component Clase4_a is

        generic (
            N : integer := N  -- Tamaño del contador
        );

        port(

            clk : in std_logic;
            reset : in std_logic;

            count : out std_logic_vector (N-1 downto 0)

        );

    end component;

```

```

signal clk : std_logic := '0';
signal reset : std_logic := '0';
signal count : std_logic_vector( N-1 downto 0 );

begin

clk <= not(clk) after 10 ns;
reset <= '1', '0' after 50 ns;

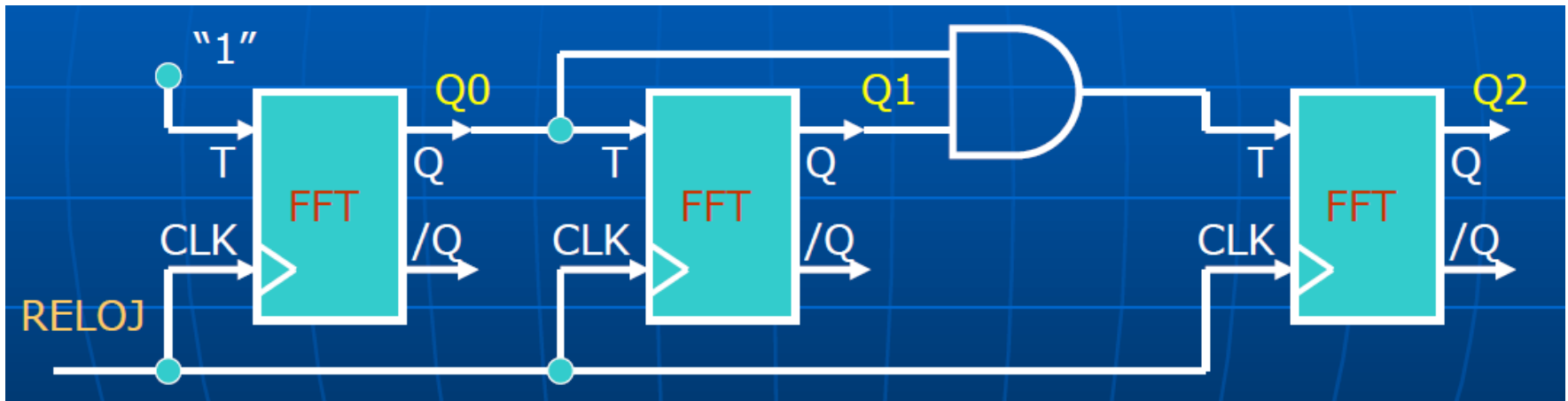
u0: Clase4_a port map (clk,reset,count);

end arch;

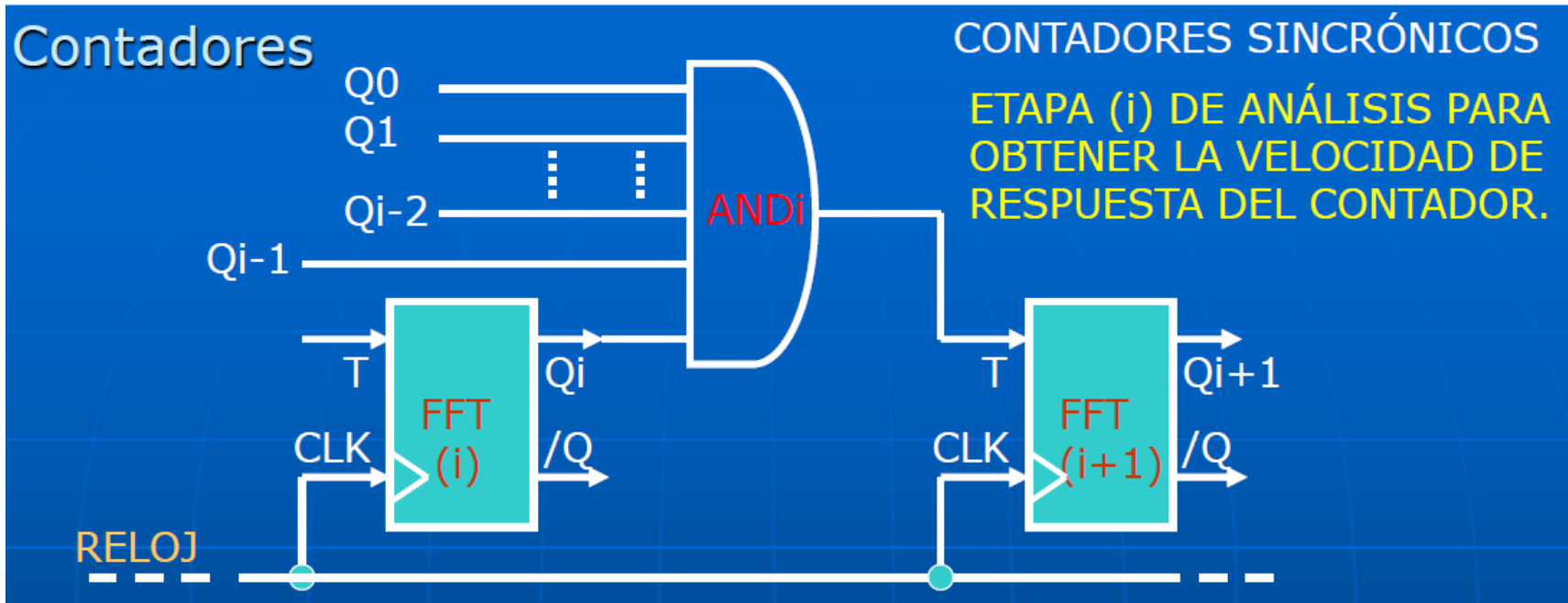
```


Contadores síncronos: 3 bits

- En general conviene que toda la lógica esté sincronizada con un reloj.
- Con algunas celdas lógicas extra y usando FFT se puede implementar un contador síncrono.



Contadores sincrónicos: N bits



$$\text{Frec. CLK (máx)} = \frac{1}{t_{pd}(\text{CLK} \rightarrow Q) + t_{\text{SET-UP}} + t_{pd_{AND}}}$$

Implementación

```
architecture counter of Clase4_b is
    signal count_aux : std_logic_vector (N-1 downto 0);
    signal vector_T : std_logic_vector (N-1 downto 0) := (others => '0');

    component FF_T is
        Port (
            T, CLK, SET, RESET : in STD_LOGIC;
            Q, Q_N              : out STD_LOGIC
        );
    end component;
```

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase4_b is
    generic
    (
        N : integer := 4
    );

    port (
        clk : in std_logic;
        reset : in std_logic;

        count : out std_logic_vector(N-1 downto 0)
    );

end Clase4_b;
```

```
begin
    process (count_aux) is
        variable aux : std_logic := '1';
    begin

        vector_T(0) <= '1';
        vector_T(1) <= count_aux(0);

        for j in 2 to N-1 loop
            for i in 0 to j-1 loop
                aux := aux and count_aux(i) ;
            end loop;
            vector_T(j) <= aux;
            aux := '1';
        end loop;

    end process;

    gen_ff:
    for i in 0 to N-1 generate
        ff: FF_T port map (vector_T(i), clk, '0', reset, count_aux(i), open);
    end generate;

    count <= count_aux;

end counter;
```

Testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity testbench is end entity testbench;

architecture test of testbench is

constant N : integer := 8;
```

```
component Clase4_b is
```

```
  generic
```

```
  (
    N : integer := 4
  );
```

```
  port (
```

```
    clk : in std_logic;
    reset : in std_logic;
```

```
    count : out std_logic_vector(N-1 downto 0)
```

```
  );
```

```
end component;
```

```
  signal clk_tb : std_logic := '0';
  signal reset_tb : std_logic := '0';
  signal count_tb : std_logic_vector (N-1 downto 0);

begin

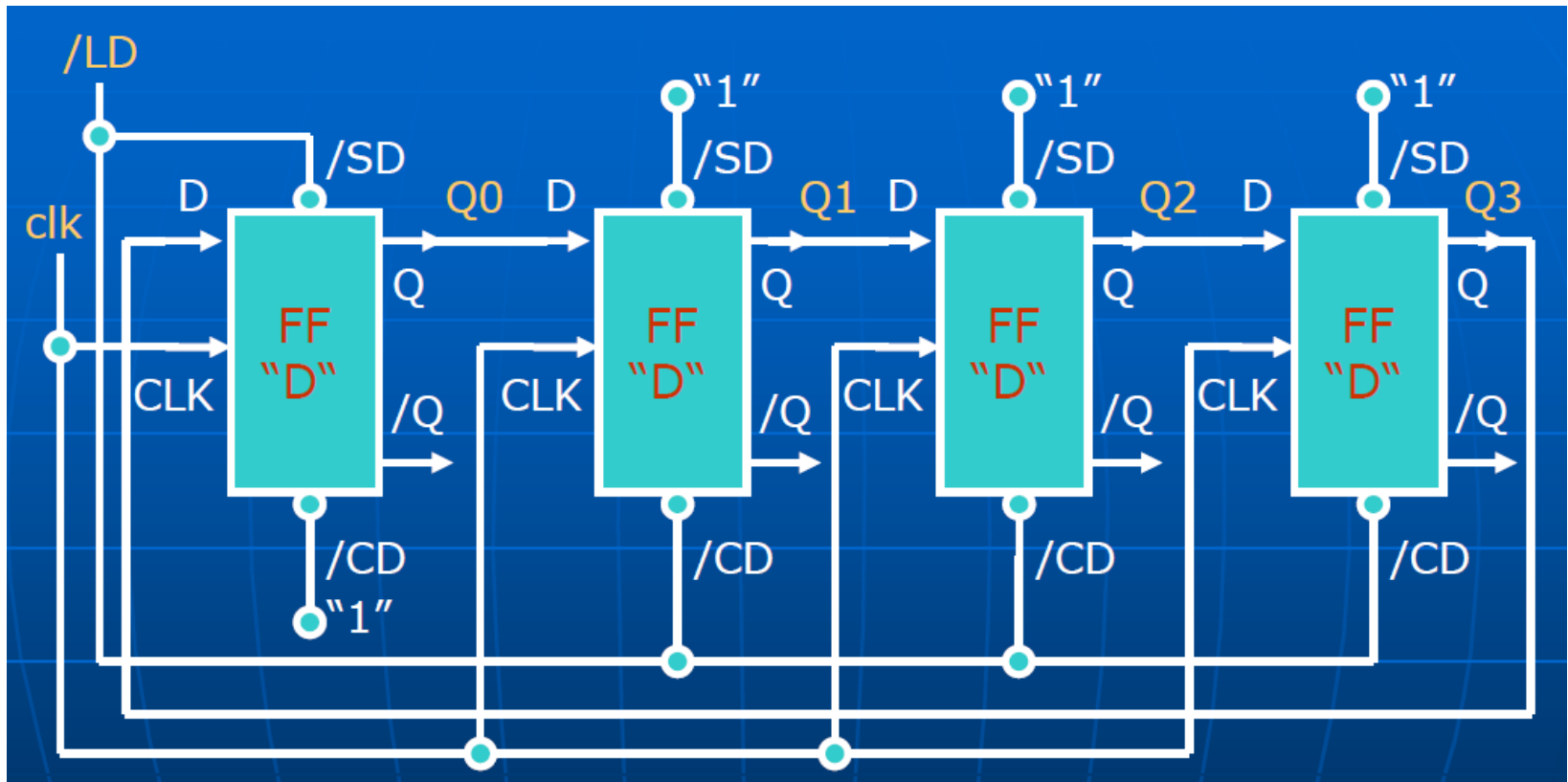
  clk_tb <= not(clk_tb) after 10 ns;
  reset_tb <= '1','0' after 40 ns;

  tb: Clase4_b generic map (N) port map (clk_tb,reset_tb,count_tb);

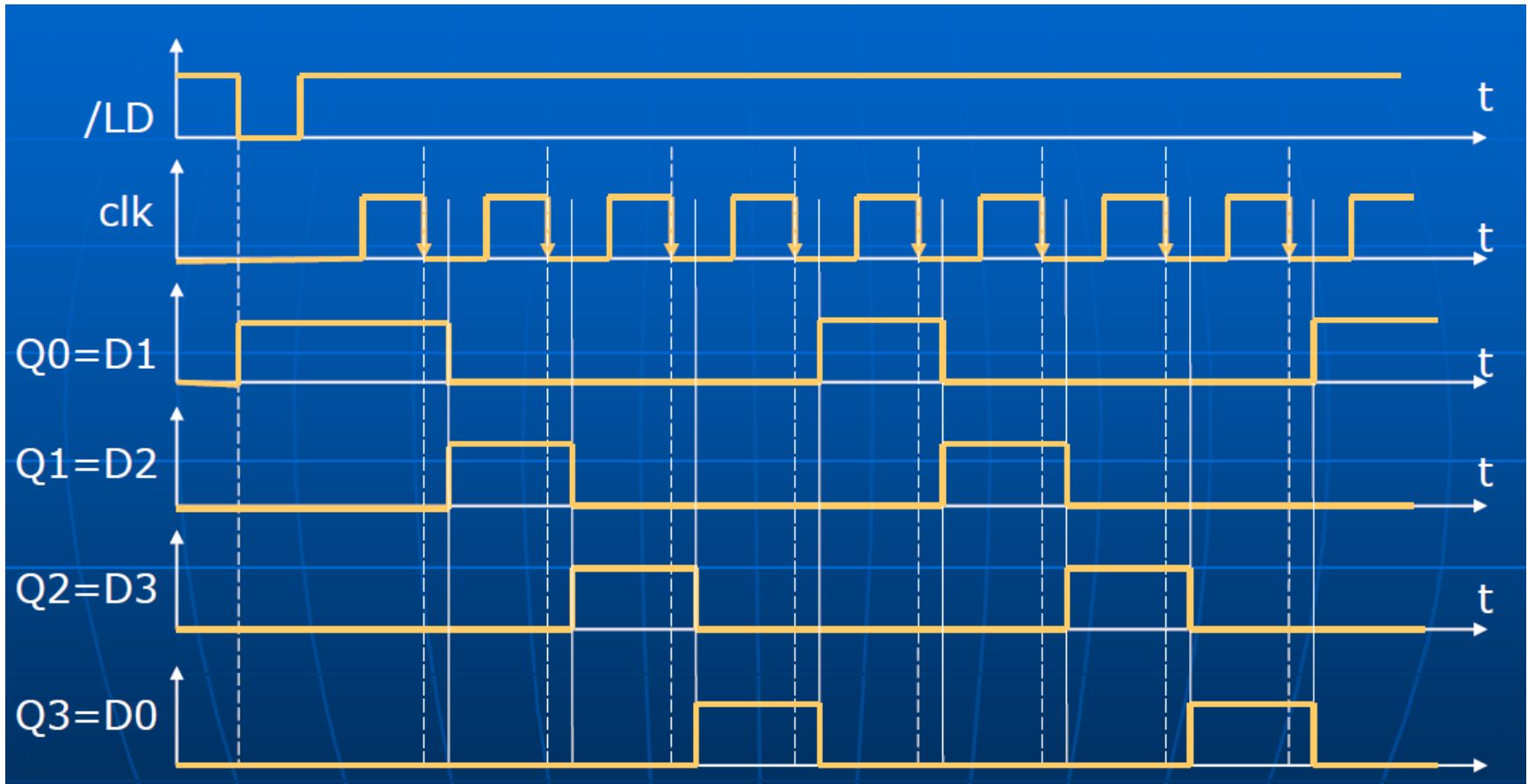
end test;
```

Contador en anillo

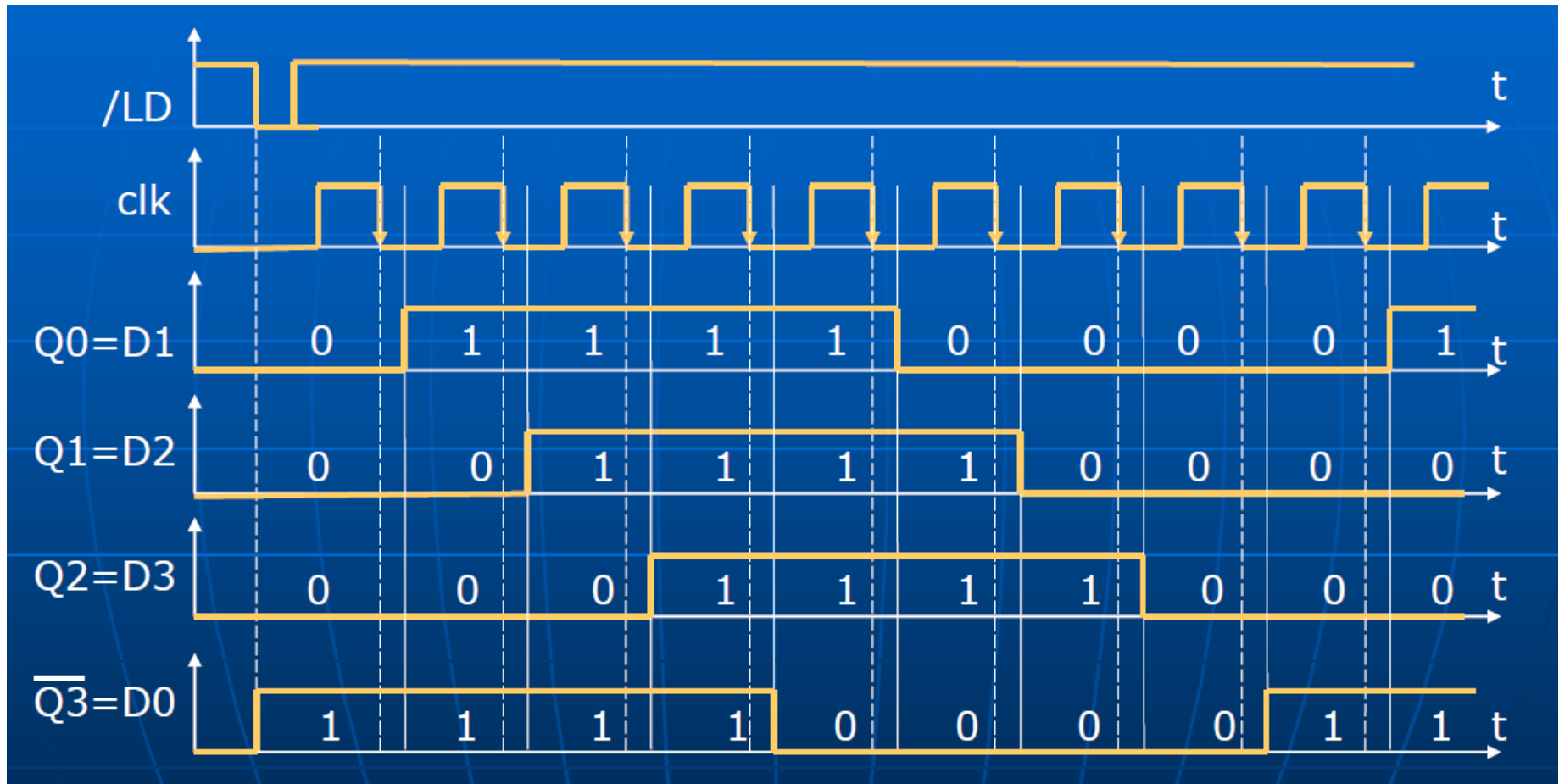
- Otros tipos de contadores se pueden implementar con FFD.
- Este empieza con un SET/RESET inicial para tener siempre un único FF en 1.



Contador en anillo

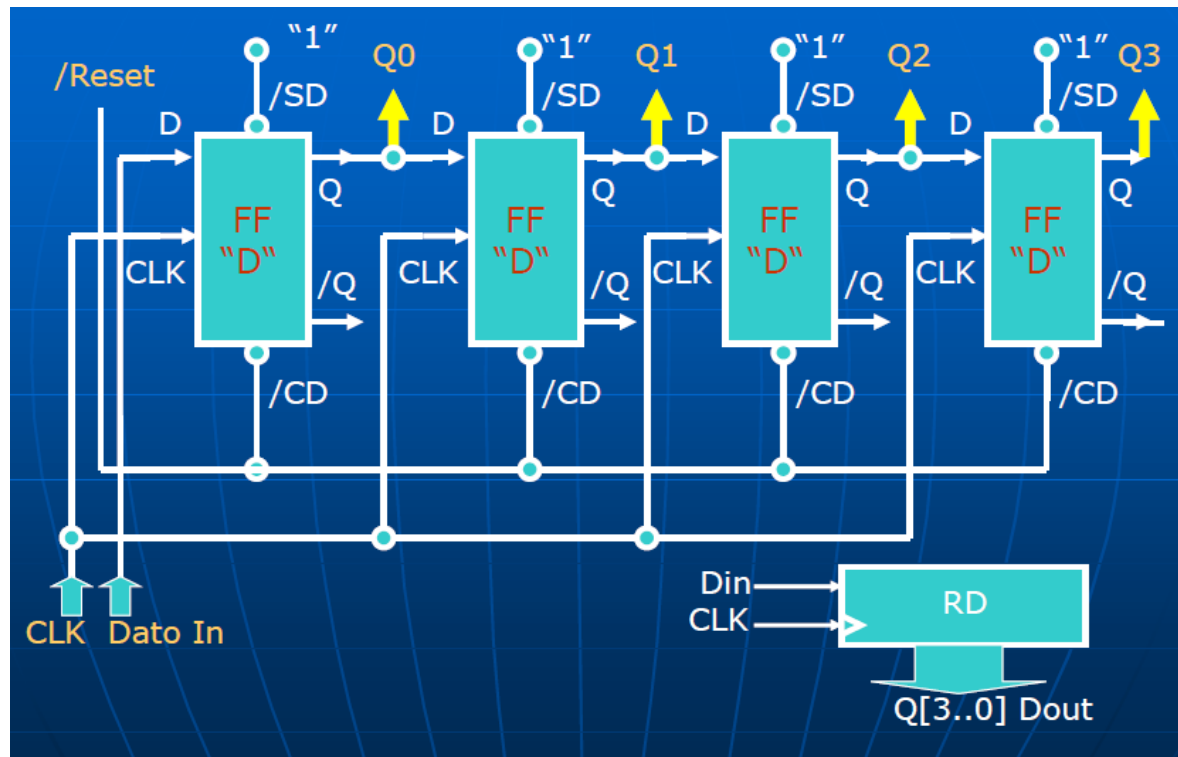


Contador Johnson



Registros de desplazamiento

- Estos circuitos transforman una señal serie en una paralela, o al revés, sincronizados por una señal de reloj.
- Por ejemplo un RD serie-paralelo de 4 bits se puede armar con 4 FFD:



Implementación RD serie-paralelo

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase4_b is

    generic
    (
        N : integer := 4
    );

    port
    (
        clk : in std_logic;
        dat_in : in std_logic;
        reset : in std_logic;
        enable : in std_logic;

        dat_out : out std_logic_vector (N-1 downto 0)
    );
end entity;
```

Implementación RD serie-paralelo

```

architecture rd_serie_paralelo of Clase4_b is

  component FF_D is
    Port (
      D, CLK, SET, RESET, ENABLE : in STD_LOGIC;
      Q, Q_N                      : out STD_LOGIC
    );
  end component;

  signal q_aux : std_logic_vector (N downto 0) := (others => '0');

begin

  ff_d_0: FF_D port map ( dat_in,clk,'0',reset,enable, q_aux(0),open );

  gen_rd:
  for i in 0 to N-2 generate

    ff_d_0: FF_D port map ( q_aux(i),clk,'0',reset,enable, q_aux(i+1),open );

  end generate;

  dat_out <= q_aux(N-1 downto 0);

end rd_serie_paralelo;

```

Testbench RD serie-paralelo

```
begin
clk_tb <= not(clk_tb) after 10 ns;
coso: Clase4_b port map (clk_tb, dat_in_tb, reset_tb, enable_tb, dat_out_tb);
process
begin

    reset_tb <= '1';
    wait for 40 ns;
    reset_tb <= '0';

    enable_tb <= '1';

    wait for 20 ns;
    dat_in_tb <= '0';

    wait for 20 ns;
    dat_in_tb <= '1';

    wait for 20 ns;
    dat_in_tb <= '1';

    wait for 20 ns;
    dat_in_tb <= '0';

    wait for 20 ns;
    enable_tb <= '0';

    wait for 40 ns;

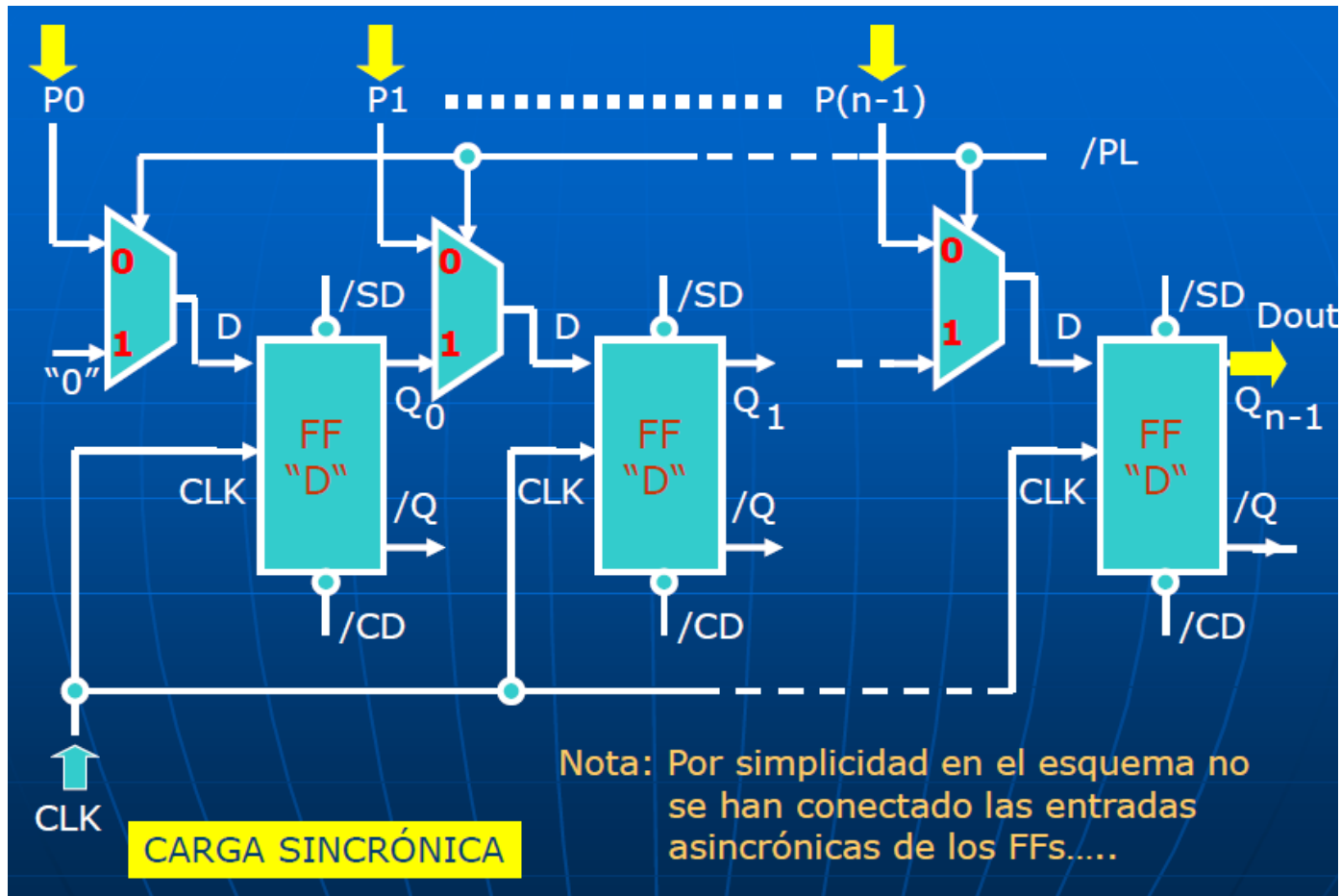
end process;

end test;
```



Por simplicidad se omite la declaración del componente y señales auxiliares

RD paralelo-serie: Carga sincrónica



RD paralelo-serie: Carga asincrónica

